

Dockerized AI Application using Docker Compose

1. Objective

To build and deploy a **multi-container AI application** using Docker and Docker Compose consisting of:

- **Frontend** – Nginx serving HTML
 - **Backend Model API** – Python Flask
 - **Database** – MongoDB
-

2. Prerequisites

Ensure the following are installed:

- Docker
- Docker Compose
- Python 3
- Git (optional)

Check installations:

```
docker --version
docker compose version
python3 --version
```

3. Project Folder Structure

Create the project directory.

```
mkdir ai-docker-app
cd ai-docker-app
```

Create the following structure:

```
ai-docker-app
├── docker-compose.yml
├── model
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
```

```
└─ frontend
   └─ index.html
      Dockerfile
```

4. Backend Model (Flask API)

Navigate to model folder.

```
mkdir model
cd model
```

app.py

```
from flask import Flask
from pymongo import MongoClient
app = Flask(__name__)
client = MongoClient("mongodb://mongodb:27017/")
db = client.testdb
@app.route("/")
def home():
    return "AI Model Running"
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

requirements.txt

```
flask
pymongo
```

Model Dockerfile

Create a file named **Dockerfile**

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

5. Frontend Setup

Go back to project folder.

```
cd ..  
mkdir frontend  
cd frontend
```

index.html

```
<!DOCTYPE html>  
<html>  
<head>  
<title>AI Docker App</title>  
</head>  
<body>  
<h1>AI Docker Application</h1>  
<p>Frontend running in Nginx container.</p>  
</body>  
</html>
```

Frontend Dockerfile

```
FROM nginx:alpine  
COPY index.html /usr/share/nginx/html/index.html
```

6. Docker Compose Configuration

Go back to project root.

```
cd ..
```

Create **docker-compose.yml**

```
version: "3"  
services:  
  mongodb:  
    image: mongo  
    container_name: mongo_db  
    ports:  
      - "27018:27017"  
  model:  
    build: ./model  
    ports:  
      - "5001:5000"  
    depends_on:  
      - mongodb  
  
  frontend:  
    build: ./frontend  
    ports:  
      - "8081:80"  
    depends_on:  
      - model
```

7. Build and Run Containers

Run the following command from the project root:

```
sudo docker compose up -build
```

This command will:

1. Build model container
2. Build frontend container
3. Pull MongoDB image
4. Start all services

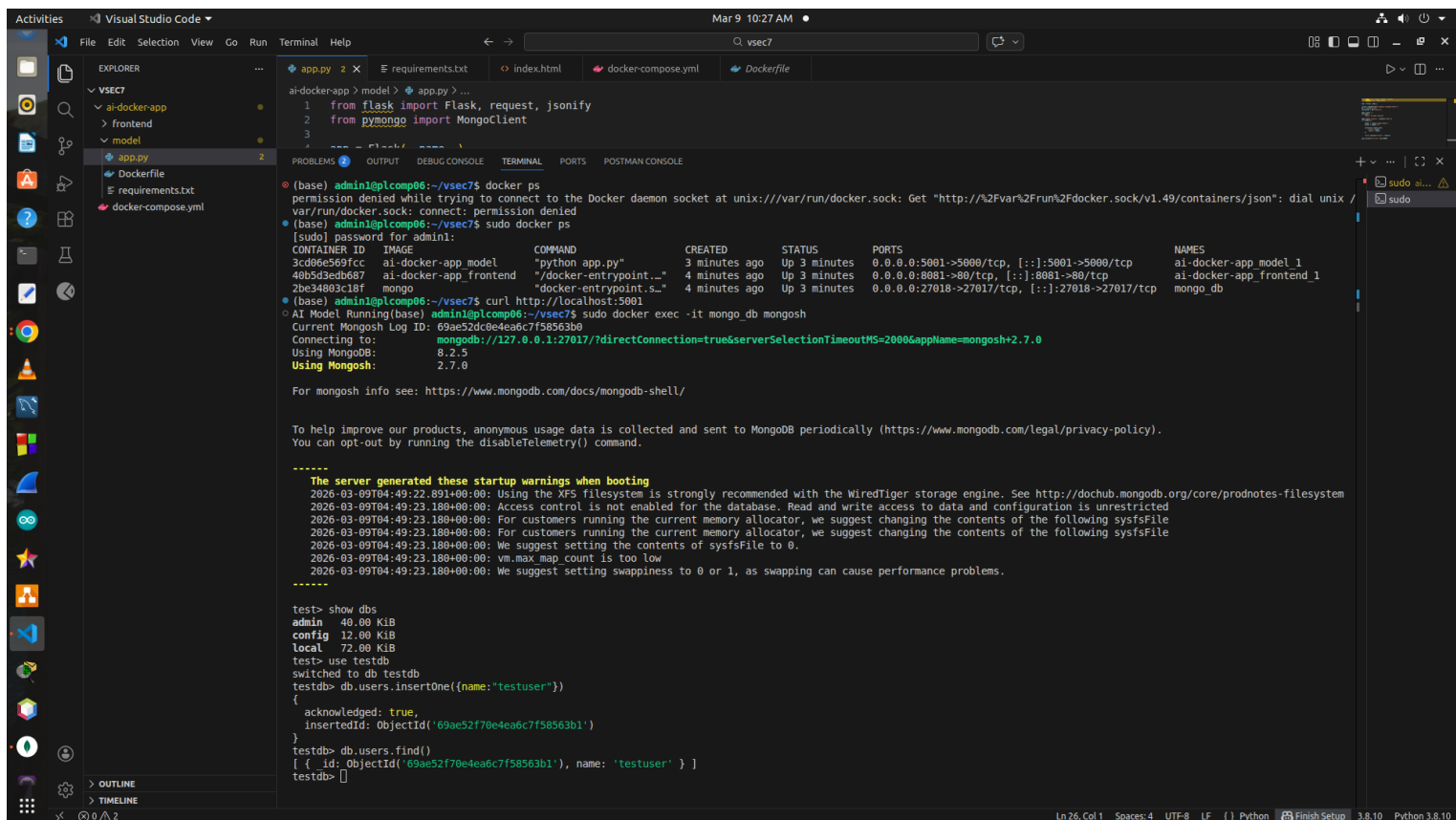
8. Verify Running Containers

Run:

```
sudo docker ps
```

Expected output:

```
ai-docker-app_model_1  
ai-docker-app_frontend_1  
mongo_db
```



```
ai-docker-app > model > app.py > ...
1 from flask import Flask, request, jsonify
2 from pymongo import MongoClient
3
4 app = Flask(__name__)
5
6 @app.route('/')
7 def index():
8     return jsonify({'message': 'Hello World'})
9
10 if __name__ == '__main__':
11     app.run(host='0.0.0.0', port=5001)
```

```
(base) admin@plcomp06:~/vsec7$ docker ps
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://localhost:2376/v1.49/containers/json": dial unix /var/run/docker.sock: connect: permission denied
(base) admin@plcomp06:~/vsec7$ sudo docker ps
[sudo] password for admin1:
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
3cd86e569fcc   ai-docker-app_model   "python app.py"         3 minutes ago   Up 3 minutes   0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp   ai-docker-app_model_1
40b5d3ed687    ai-docker-app_frontend "/docker-entrypoint.s..." 4 minutes ago   Up 3 minutes   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp       ai-docker-app_frontend_1
2be34803c18f    mongo             "docker-entrypoint.s..." 4 minutes ago   Up 3 minutes   0.0.0.0:27018->27017/tcp, [::]:27018->27017/tcp   mongo_db
(base) admin@plcomp06:~/vsec7$ curl http://localhost:5001
{"message": "Hello World"}
AI Model Running(base) admin@plcomp06:~/vsec7$ sudo docker exec -it mongo_db mongosh
Current Mongosh Log ID: 69ae52dc0e4e6c7f58563b0
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      8.2.5
Using Mongosh:      2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-03-09T04:49:23.891+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-03-09T04:49:23.180+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-03-09T04:49:23.180+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-03-09T04:49:23.180+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-03-09T04:49:23.180+00:00: We suggest setting the contents of sysfsFile to 0.
2026-03-09T04:49:23.180+00:00: vm.max_map_count is too low
2026-03-09T04:49:23.180+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
-----

test> show dbs
admin   40.00 KiB
config 12.00 KiB
local  72.00 KiB
test> use testdb
switched to db testdb
testdb> db.users.insertOne({name:'testuser'})
{
  acknowledged: true,
  insertedId: ObjectId('69ae52f70e4e6c7f58563b1')
}
testdb> db.users.find()
[ { _id: ObjectId('69ae52f70e4e6c7f58563b1'), name: 'testuser' } ]
testdb>
```

9. Testing the Application

Test Frontend

Open browser:

<http://localhost:8081>

Expected output:

AI Docker Application
Frontend running in Nginx container.

Test Backend API

Run:

curl <http://localhost:5001>

Expected output:

AI Model Running

Test MongoDB

Access MongoDB container:

```
sudo docker exec -it mongo_db mongosh
```

Show databases:

```
show dbs
```

Create database:

```
use testdb
```

Insert data:

```
db.users.insertOne({name:"testuser"})
```

View data:

```
db.users.find()
```

Exit MongoDB:

```
Exit
```

10. Check Container Logs

Frontend logs:

```
sudo docker logs ai-docker-app_frontend_1
```

Model logs:

```
sudo docker logs ai-docker-app_model_1
```

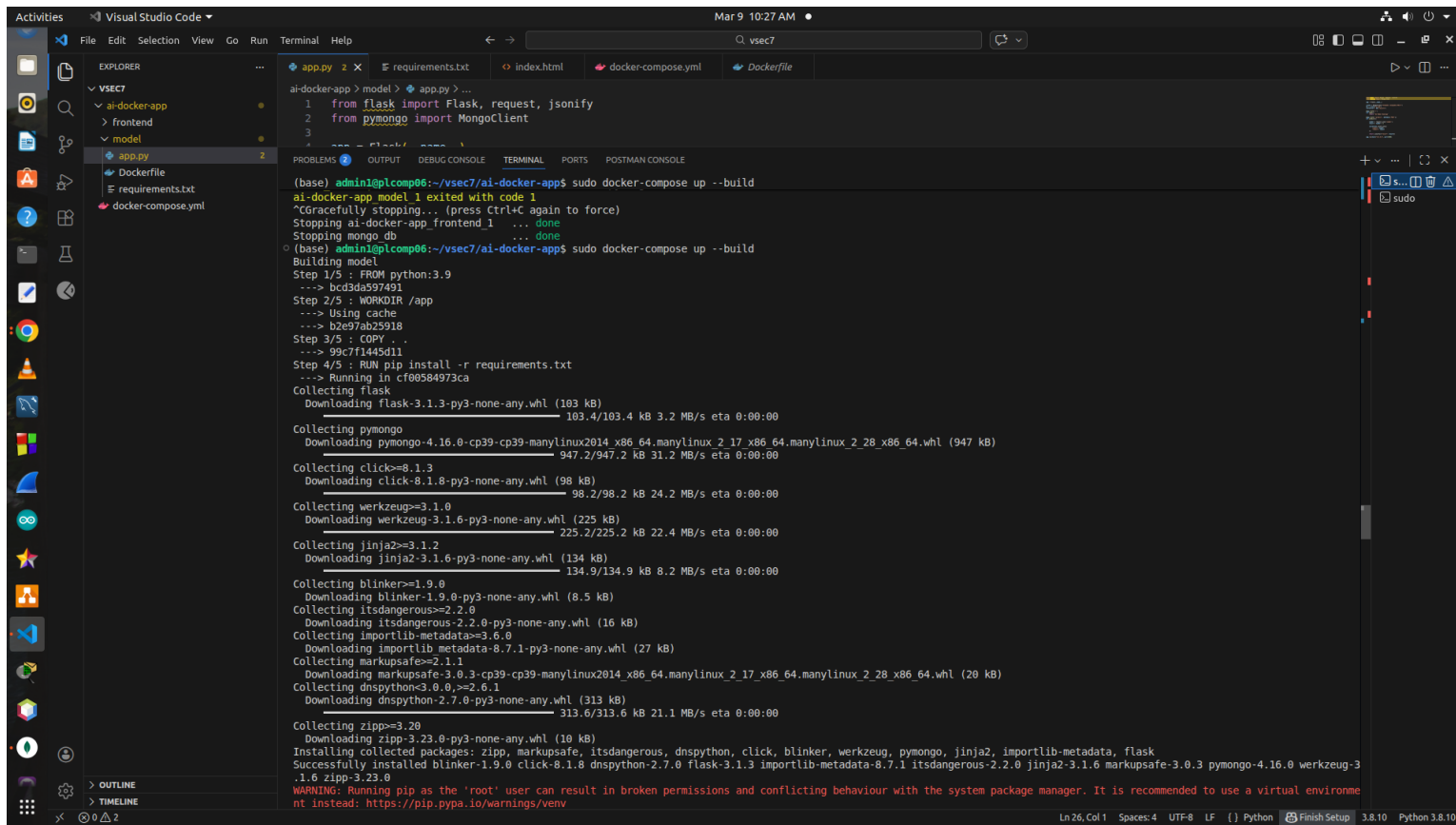
MongoDB logs:

```
sudo docker logs mongo_db
```

11. Stopping Containers

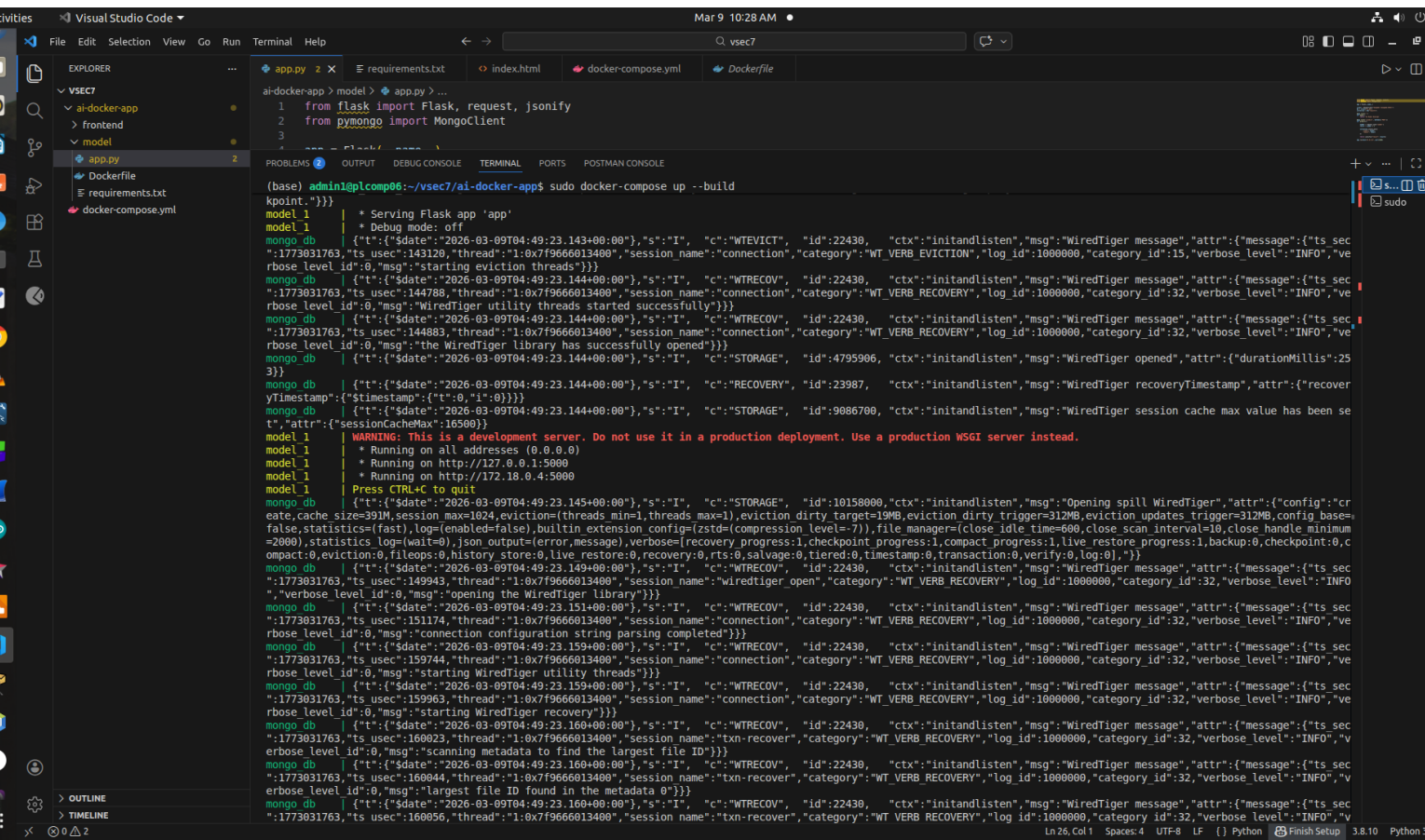
To stop all services:

`docker compose down`



The screenshot shows a Visual Studio Code window with a terminal open. The terminal output shows the process of stopping and rebuilding Docker containers. The user runs `sudo docker-compose up --build` to rebuild the containers. The output shows the following steps:

- Building model
- Step 1/5: FROM python:3.9
- Step 2/5: WORKDIR /app
- Step 3/5: COPY . .
- Step 4/5: RUN pip install -r requirements.txt
- Collecting flask
- Collecting pymongo
- Collecting click
- Collecting werkzeug
- Collecting Jinja2
- Collecting blinker
- Collecting itsdangerous
- Collecting importlib-metadata
- Collecting MarkupSafe
- Collecting dnspython
- Collecting zipp
- Successfully installed blinker-1.9.0 click-8.1.8 dnspython-2.7.0 flask-3.1.3 importlib-metadata-8.7.1 itsdangerous-2.2.0 Jinja2-3.1.6 MarkupSafe-3.0.3 pymongo-4.16.0 werkzeug-3.1.6 zipp-3.23.0
- WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv



The screenshot shows a Visual Studio Code window with a terminal open. The terminal output shows the process of starting Docker containers. The user runs `sudo docker-compose up --build` to build and start the containers. The output shows the following steps:

- Building model
- Step 1/5: FROM python:3.9
- Step 2/5: WORKDIR /app
- Step 3/5: COPY . .
- Step 4/5: RUN pip install -r requirements.txt
- Collecting flask
- Collecting pymongo
- Collecting click
- Collecting werkzeug
- Collecting Jinja2
- Collecting blinker
- Collecting itsdangerous
- Collecting importlib-metadata
- Collecting MarkupSafe
- Collecting dnspython
- Collecting zipp
- Successfully installed blinker-1.9.0 click-8.1.8 dnspython-2.7.0 flask-3.1.3 importlib-metadata-8.7.1 itsdangerous-2.2.0 Jinja2-3.1.6 MarkupSafe-3.0.3 pymongo-4.16.0 werkzeug-3.1.6 zipp-3.23.0
- WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv